Typography

Strict golden-ratio scaling makes body text too small for comfortable reading, so I'd adapt the scale:

Element	Recommended size
Hero title	82px
Section title	51px
Project title	31px
Subheading	20px
Body text	18px
Small text	14px
Technical labels	12px
Buttons	22px

Color Pallette

Project: Trove — Interactive Portfolio Experience

1. Project Overview

Build a fully interactive, frontend-only simulation of **Trove**, a tangible interactive artifact that allows users to explore the stories and meanings behind physical objects.

This application will initially function as a standalone component, but it must be designed for seamless integration into my existing personal developer portfolio.

The portfolio uses an **architectural blueprint-inspired visual identity**, with dark navy backgrounds, fine technical grid lines, orange accents, engineering annotations, and precise typography.

Trove should inherit this visual identity while retaining the physical artifact's playful, child-friendly appearance.

Core concept

"Every object has a story."

Visitors interact with a front-facing illustration of the physical Trove artifact. Three objects appear beside the artifact:

1. A pen
2. A Burmese postage stamp
3. A rocket keychain

Visitors can drag any object onto Trove's white scanning patch.

Trove then simulates scanning the object and produces an animated conversational response explaining the object's purpose, cultural relevance, or story.

The experience should feel like interacting with a real physical prototype through a browser.

---

2. Technology Requirements

Use the following technologies where compatible with the existing portfolio:

- React with TypeScript
- Tailwind CSS
- Motion for React for layout and transition animations
- CSS keyframes for scanning effects
- Pointer Events API for desktop and mobile drag-and-drop
- SVG for technical annotations and blueprint details

No backend is required.

Do not implement actual computer vision, image recognition, authentication, databases, or external AI API calls.

The responses are predefined simulations.

Integration requirements

The standalone component should:

- Be self-contained and reusable.
- Work inside an existing React application.
- Avoid global CSS conflicts.
- Avoid modifying the portfolio's existing navigation or layout.
- Support embedding inside an expandable project section.
- Scale responsively to the width of its parent container.
- Not require a full-screen page or iframe.
- Avoid unnecessary dependencies.
- Support lazy loading to reduce initial portfolio load time.

Create a main component named:

`TroveInteractive.tsx`

Organize its supporting components, configuration, and assets into an isolated `trove` directory.

---

3. Visual Design System

Maintain the portfolio's existing blueprint visual identity.

Color palette

Purpose	Color
Main background	#082452

Blueprint blue	#0C3F9B
Secondary blue	#174987
Technical white	#F4F8FF
Cyan highlight	#67DDF5
Orange accent	#FF791E
Muted technical text	#A9C9ED

Use a deep navy background with subtle, evenly spaced blueprint grid lines.

The grid should remain visible but never interfere with readability.

Use thin white or pale-blue strokes for technical annotations.

Use orange sparingly to highlight active elements, draggable objects, and important interactions.

Typography

Use the portfolio's existing font family.

For technical annotations, use a monospace font.

Suggested hierarchy:

- Main heading: 32–40px desktop, 26–30px mobile
- Section labels: 11–13px
- Body copy: 15–17px
- Technical annotations: 10–12px

Avoid unnecessarily large headings or excessive text.

The main emphasis should be on the interactive physical artifact.

---

4. Main Application Layout

The application should have three primary areas:

A. Trove physical artifact

B. Three freely draggable objects

C. Trove conversational response

Desktop layout

Use a centered, spacious composition.

The Trove artifact should occupy approximately 65–75% of the main interactive area's width.

The three object illustrations should appear to its right, arranged vertically or in a loosely staggered arrangement.

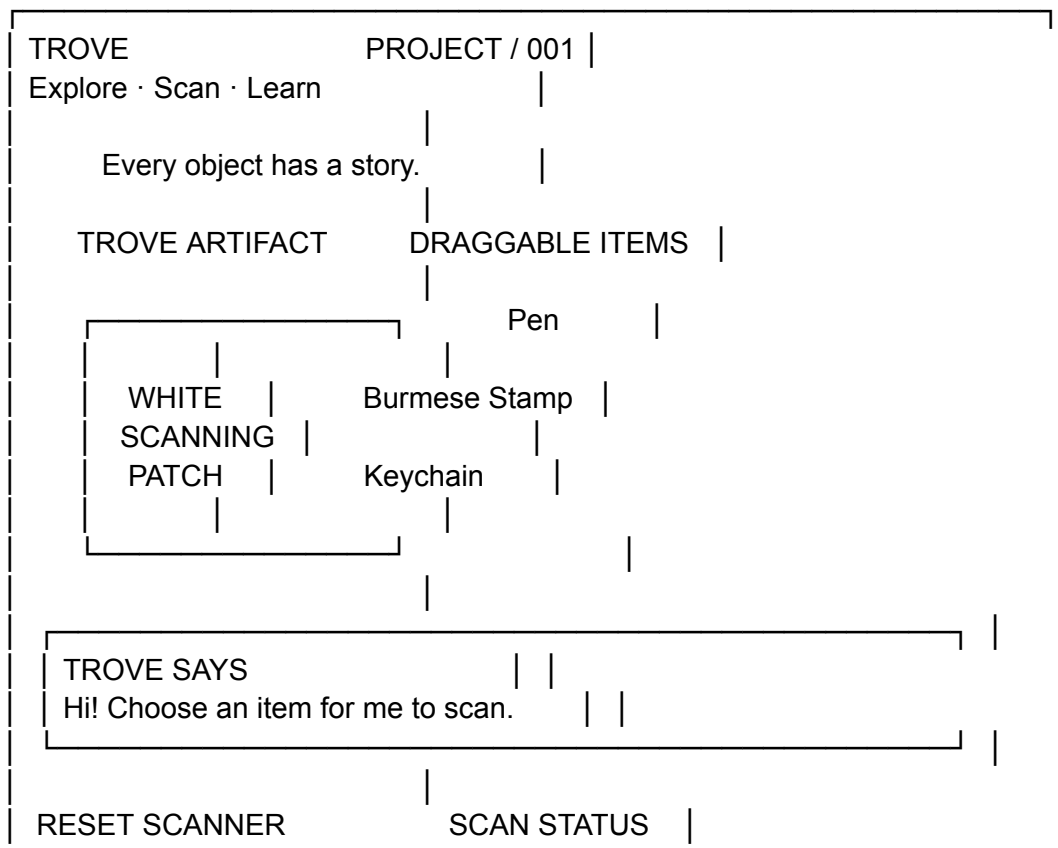
The objects must not be enclosed in rectangular cards.

They should look like physical items floating in a design workspace.

Each item may have a small technical label positioned nearby.

Below the artifact and objects, display a horizontal conversational area spanning the available width.

Suggested hierarchy:



└──┘

Mobile layout

On smaller screens:

1. Display the Trove artifact at the top.
2. Display the three objects underneath in a horizontal, freely arranged row.
3. Place the conversational response below the objects.
4. Preserve a usable scanning patch.
5. Support touch dragging and tap-to-scan.
6. Avoid horizontal overflow.

The design must feel native to mobile rather than simply shrinking the desktop version.

---

5. Trove Physical Artifact

Use the **provided Trove blueprint artwork** as the central visual asset.

Do not replace it with an emoji, generic scanner, robot, or unrelated illustration.

The artifact features:

- A soft tunnel-shaped body.
- Blue fabric with playful doodle patterns.
- Gray felt surfaces.
- Orange accent elements.
- A prominent white rectangular scanning patch.
- An orange cross on the interior floor.

Preserve the artifact's proportions and visual details.

Interactive scanning region

Position a transparent interactive HTML drop zone over the white rectangular patch.

The drop zone must align precisely with the patch at different screen sizes.

Use a responsive relative-positioning system.

Do not hardcode screen-pixel positions that break when the component resizes.

The overlay should have the same approximate dimensions and proportions as the white patch.

When idle, the original white patch should remain visible.

When an object is dragged over it:

- The scanning patch glows cyan.
- A subtle dashed outline becomes visible.
- The scanner indicates that the object can be dropped.
- The artifact should not jump or resize.

Use a clearly visible active state without obscuring the artifact.

---

6. Three Interactive Objects

Use the three **previously generated blueprint-style images**.

Do not substitute emojis, generic icons, or newly generated placeholder images when the supplied assets are available.

Object 01 — Pen

A blue pen with white technical details and orange accents.

Behavior:

- Can be dragged onto Trove's scanning patch.
- Can be selected with a click or tap.
- Has a small hover or selected-state animation.

Trove's response:

"You found a pen! A simple writing tool can capture ideas, preserve memories, and turn imagination into stories. What would you write with it?"

Object 02 — Burmese postage stamp

A Myanmar-inspired postage stamp with architectural imagery, decorative edges, and a blue-and-orange design.

Behavior follows the same interaction model as the pen.

Trove's response:

"This is a Burmese postage stamp! Stamps do more than send letters. They can preserve history, celebrate culture, and tell stories about a place and its people."

Object 03 — Rocket keychain

A blue rocket-shaped keychain with a metal ring and orange details.

Behavior follows the same interaction model.

Trove's response:

"This is a rocket keychain! Keychains help us keep important keys together. They can also remind us of special places, favorite interests, and meaningful experiences."

Object styling

Objects should appear as isolated illustrations.

Do not surround them with cards, boxes, or large solid backgrounds.

Each object should have:

- A subtle floating animation while idle.
- A small hover scale effect.
- A grabbing cursor while dragged.
- A soft shadow or glow when selected.
- A technical label and identifier displayed nearby.

Do not allow animations to make the interface feel chaotic.

---

7. Drag-and-Drop Functionality

Implement functional pointer-based drag-and-drop.

The interaction should work with:

- Desktop mouse
- Laptop trackpad
- Mobile touch
- Tablet touch

Interaction sequence

Step 1 — Initial state

All three objects are visible.

Trove displays:

"Hello! I'm Trove. Every object has a story. Choose something for me to scan!"

The scanner is idle.

Step 2 — Object selection

When the user clicks or taps an object:

- Mark the object as selected.
- Give it a subtle glow.
- Display a short instruction telling the user to place it onto the scanner.

Step 3 — Dragging

When the user begins dragging:

- The selected item follows the pointer.
- The object appears slightly enlarged.
- The original location remains visually understandable.
- The scanner highlights when the pointer enters its drop region.

Step 4 — Drop detection

When the user releases the object inside the scanning patch:

- Verify that the release position intersects the drop zone.
- Identify which of the three objects was dropped.
- Begin the simulated scanning sequence.

If the object is released outside the drop zone, animate it back to its original position.

Step 5 — Scanning

The object appears centered over the white scanning patch.

Display a moving cyan scanning line over the object.

Use a short sequence such as:

"Object detected..."

"Analyzing object..."

"Scan complete!"

The complete scanning animation should last approximately 1–1.5 seconds.

Step 6 — Response

After the scan finishes:

- Display the object's predefined response in the conversation area.
- Animate the message appearing.
- Update the status indicator to "SCAN COMPLETE."
- Preserve the scanned object visually until another object is selected or the demo is reset.

Touch accessibility

Because drag gestures can be difficult on smaller screens, provide an alternative:

Tap an item → tap the white scanning patch → begin scanning.

Keyboard users should also be able to select an object and activate the scanner.

---

8. Trove Conversational Interface

The conversational interface must appear **below the artifact and draggable items**, not beside the Trove artifact.

Use a full-width response area styled like a speech bubble or conversational panel.

The chat panel should feel integrated into the blueprint environment.

Chat appearance

Use:

- A light background for the speech bubble.
- Dark navy text.
- A small "TROVE SAYS" label.
- An optional speaker icon.
- A subtle orange or cyan accent.

The response should animate into view using a short fade or text reveal.

Avoid excessive typewriter delays.

Conversation behavior

For the first version, support one active response at a time.

The user is not typing messages into a chatbot.

Trove responds automatically when an object is scanned.

Do not create a fake text input field or imply that the system is connected to a live AI model.

Optional enhancement:

Use the browser's SpeechSynthesis API to let visitors hear Trove read the response aloud.

Audio must remain off by default and require a user action to play.

Provide a replay and stop control if speech is implemented.

---

9. Scanning Animation

Create a subtle but polished scanning effect.

The scanning animation should include:

- Cyan horizontal laser line moving across the patch.
- A brief glow around the patch.
- A detected object image appearing on the patch.
- A scanning status update.
- A completed-state indicator.

Avoid heavy effects that cover the artifact.

The white patch must remain recognizable.

Respect `prefers-reduced-motion` by replacing motion-heavy effects with immediate state changes.

---

10. Reset Functionality

Include a small reset control underneath the conversation area.

Label:

"RESET SCANNER"

When activated:

- Clear the selected object.
- Remove the object from the scanning patch.
- Restore the initial conversation.
- Reset status to idle.
- Return all objects to their starting positions.
- Cancel any active speech or scanning effects.

The reset function must also work while an animation is in progress.

---

11. Blueprint Engineering Details

Preserve the portfolio's engineering identity through restrained visual details.

Recommended annotations:

- `TROVE / INTERACTIVE ARTIFACT`
- `FIG. 01 / FRONT VIEW`
- `SCAN AREA / ACTIVE`
- `OBJECT RECOGNITION / SIMULATED`
- `PROTOTYPE 001`
- `FRONT-END EXPERIENCE`

Use thin technical borders and small label markers.

Add subtle background drafting lines, dimension indicators, or coordinate markers where appropriate.

Keep decoration secondary to the interface.

Do not add speculative dimensions or measurements that have not been provided.

---

12. Portfolio Integration

The standalone application will eventually appear inside the Trove project entry of my portfolio.

The portfolio's project section should support expandable project cards.

Collapsed project state

Display:

TROVE

Tangible Interaction · Human-Centered Design

A short introduction to the project and an "EXPLORE PROJECT" control.

Expanded project state

Clicking "EXPLORE PROJECT" should:

1. Expand the project container vertically.
2. Smoothly push subsequent project cards downward.
3. Reveal the interactive Trove simulation.
4. Preserve the surrounding portfolio layout.
5. Allow visitors to interact with the scanner without leaving the page.

Avoid full-screen modals for the main experience.

Project interaction lifecycle

When expanded:

- Initialize the simulation.
- Show the idle state.
- Allow interaction immediately.

When collapsed:

- Stop speech playback.
- Cancel active drag operations.
- Cancel pending scanning animations.
- Prevent ongoing effects from running invisibly.

Prefer preserving the last completed scan until the user explicitly resets or the portfolio is reloaded.

Reusability

Build Trove as a modular component that can later be included in the portfolio using an interface similar to:

<TroveInteractive />

The parent portfolio component should control whether the project is expanded.

Trove should control only its own interactive behavior.

---

13. Component Architecture

Use a clean, maintainable component structure.

Suggested directory:

```
src/
  components/
    projects/
      trove/
        TroveInteractive.tsx
        TroveArtifact.tsx
        DraggableObject.tsx
        ScannerPatch.tsx
        TroveConversation.tsx
        troveData.ts
        trove.css

public/
  assets/
    trove/
      trove-artifact.png
      pen.png
      burmese-stamp.png
      keychain.png
```

Component responsibilities

TroveInteractive

Owns the main application state and coordinates interactions.

TroveArtifact

Renders the physical artifact image.

DraggableObject

Handles object selection, dragging, and returning to origin.

ScannerPatch

Manages the interactive scanning region.

TroveConversation

Displays Trove's response and optional audio controls.

troveData

Stores object identifiers, image paths, labels, and predefined responses.

Use typed data structures.

Keep object content separate from UI logic.

---

14. Required Application States

Implement these explicit states:

State	Description
Idle	No object selected
Selected	An object is selected
Dragging	Object follows the pointer
Hovering	Object is over the scanner
Scanning	Recognition animation plays
Completed	Object identified and response displayed

Handle rapid interactions correctly.

A user should not be able to accidentally start overlapping scanning operations.

If another object is selected during scanning, cancel the previous operation cleanly or temporarily disable scanning until completion.

Avoid stale timers and race conditions.

---

15. Responsive and Accessibility Requirements

Support common viewport sizes:

- 1440px desktop
- 1024px tablet
- 768px small tablet
- 390px mobile
- 320px narrow mobile

Requirements:

- No clipped assets.
- No unintended horizontal scrolling.
- Consistent scanner alignment.
- Touch-friendly target sizes.
- Meaningful image alt text.
- Visible focus indicators.
- Keyboard-selectable objects.
- Announced scan results through an accessible live region.
- Readable text contrast.
- Reduced-motion support.

Objects should be draggable without interfering with normal page scrolling outside the interaction area.

---

16. Performance Requirements

Optimize the component for portfolio use.

- Use provided static visual assets.
- Compress large images without visible quality loss.
- Lazy-load the interactive component if appropriate.
- Avoid unnecessary rerenders while dragging.
- Use requestAnimationFrame or performant pointer updates for moving objects.
- Remove event listeners and cancel animation tasks during cleanup.
- Avoid layout shifts when images load.
- Do not request camera or microphone permissions.

The component must work without an internet connection once its local assets and dependencies are loaded.

---

17. Deliverables

Produce:

1. A fully working React/TypeScript frontend implementation.
2. Reusable Trove components.
3. Integrated blueprint styling.
4. Responsive desktop and mobile layouts.
5. Functional drag-and-drop scanning.
6. Tap-to-scan and keyboard interaction.
7. Three generated object assets integrated into the experience.
8. Animated scanning and predefined object responses.
9. Reset functionality.
10. Clear instructions for embedding the component in the portfolio.

If editing my existing portfolio repository, inspect its structure and styling conventions before creating files.

Reuse its existing build tools and dependencies whenever practical.

Do not overwrite unrelated portfolio components.

---

18. Acceptance Criteria

The implementation is complete only when the following are true:

- All three supplied object images appear without square card containers.
- The supplied Trove artwork appears as the main artifact.
- The white scanning patch is correctly aligned and interactive.
- Each object can be dragged and dropped onto the patch on desktop.
- Mobile visitors can scan using touch drag or tap-to-scan.
- The scanner animates and identifies the correct selected object.
- Each object triggers its own predefined response.
- The response panel appears below the artifact and objects.
- Reset reliably returns the interface to its initial state.
- The component does not break the surrounding portfolio.
- The layout works across desktop, tablet, and mobile.
- There are no console errors, missing assets, or broken event handlers.

Test all major interactions rather than assuming they work.

---

19. Design Quality Expectations

The experience should feel like a professionally designed, interactive portfolio exhibit.

It should combine three qualities:

Playfulness: The physical artifact and objects should feel approachable and tactile.

Technical precision: The blueprint-style visual system should feel deliberate and consistent.

Clarity: Visitors should immediately understand how to interact with the demo without reading extensive instructions.

Avoid turning the interface into a traditional dashboard, a generic chatbot, or an e-commerce product gallery.

The emphasis should remain on the physical Trove artifact and the simple interaction of discovering the stories behind objects.

Final instruction

Implement the complete frontend experience, not just a static mockup.

Use the provided Trove artwork and the three generated object images as actual application assets.

Prioritize visual fidelity to those assets, smooth interactions, accessibility, responsive layout, and clean component architecture.

The finished experience should be usable as a standalone demo and ready to embed into an expandable project section of my existing personal portfolio.

The AI when detecting Keychain should say “ This keychain was given to Nyan Lin Zaw when he visited the Museum of Arts and Science in Chicago during the TEI conference.

When detecting stamps it should say “ This stamp was brought by Nyan Lin Zaw when he came to the US from his country as a sentimental artefact.”

When detecting pen it should say “ This pen was used by Nyan Lin Zaw in his exams throughout the year. It is a pen his parents bought specially to be used in exams.”

Assets:

Trove	
Pen	

Keychain	
Stamp	

Project: NAND Lab — Interactive Breadboard Logic Game

1. Project Overview

Build a fully interactive, frontend-only educational game called **NAND Lab**, where visitors construct a working AND gate using two NAND gates on a virtual breadboard.

The project will initially function as a standalone application but must be designed as a reusable component that can be embedded within my existing developer portfolio.

The experience should combine:

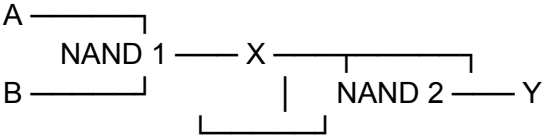
- Interactive digital logic circuit construction
- Breadboard-inspired visual design
- Drag-and-drop and click-to-connect wiring
- Real-time logic simulation
- Input switches and LED feedback
- Circuit validation and a mission checklist
- Educational explanations and hints

The design must follow my portfolio's **engineering blueprint aesthetic**, using dark navy backgrounds, white technical lines, subtle grid patterns, and orange circuit connections.

Main objective

The user must build an AND gate using two NAND gates.

The target circuit is:



X = NOT(A AND B)

Y = NOT(X AND X)

Y = A AND B

A completed circuit must produce the following truth table:

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

The challenge is complete when the player has correctly wired the two NAND gates and verified all four input combinations.

---

2. Technology Requirements

Use the following technologies when compatible with the existing portfolio:

- React
- TypeScript
- Tailwind CSS
- SVG for wires, gates, breadboard holes, and circuit annotations
- Motion for React for animations
- Pointer Events API for mouse, touch, and pen input

No backend is required.

Do not implement authentication, databases, external APIs, or server-side logic.

All circuit simulation and validation must happen entirely in the browser.

Standalone and embedded support

The game must work in two contexts:

Standalone mode: A complete educational game accessible from its own route.

Portfolio mode: A reusable component embedded within an expandable project container.

The game must not depend on a specific page width or require full-screen display.

---

3. Visual Design

Follow the architectural blueprint style used throughout my portfolio.

Color palette

Element	Color
Main background	`#082452`
Blueprint background	`#0C3F9B`
Secondary panels	`#123D73`
Technical lines	`#F4F8FF`
Grid lines	`#4979CE`
Active wires	`#FF791E`
Interactive highlights	`#67DDF5`
Muted labels	`#A9C9ED`

Design characteristics

- Dark navy background
- Subtle square engineering grid
- Thin white technical outlines
- Orange jumper wires
- Technical labels and annotations
- Monospace text for engineering information
- Precise alignment
- Minimal decoration
- Smooth, restrained animations

The interface should resemble an interactive digital circuit engineering blueprint.

Do not design it as a cartoon game, generic dashboard, or conventional form-based application.

---

4. Main Application Layout

Create a responsive interface containing four primary sections.

A. Header

Display:

NAND LAB

Constructing an AND Gate Using NAND Gates

Include a small label:

```
DIGITAL LOGIC / INTERACTIVE SIMULATION
```

B. Interactive Breadboard

The main centerpiece should be a large interactive breadboard.

Display:

- Breadboard holes arranged in rows and columns
- Power and ground rails
- Two NAND gate components
- Input A
- Input B
- Output LED
- Connection terminals
- Orange wires representing completed connections

The breadboard should occupy most of the available desktop width.

C. Circuit Information Panel

Show:

- Current input values
- First NAND gate output

- Second NAND gate output
- Expected AND output
- Circuit validation status
- Truth table progress

D. Controls and Instructions

Include:

- Place gates
- Reset circuit
- Validate circuit
- Run truth table
- Show hint
- Mission checklist

---

5. Breadboard Interaction

Implement an interactive SVG-based circuit workspace.

The breadboard should be a real interactive interface rather than a static background image.

Component placement

Provide two NAND gate components:

- NAND 1
- NAND 2

Players should place the gates onto designated positions on the breadboard.

When a gate is placed:

- Snap it into position.
- Reveal its input and output connection pins.
- Mark the corresponding mission step as complete.

The two gates should remain repositionable until wiring begins, or provide clearly designated placement locations.

Simplified simulation model

Use an educational logic-level representation rather than a physically accurate electrical simulation.

Each NAND gate has:

- Input 1
- Input 2
- Output

The first NAND gate combines A and B.

The second NAND gate inverts the first NAND gate's output by connecting that signal to both of its inputs.

The logic must be calculated from the player's actual connections.

Do not simply display the expected output regardless of how the gates are wired.

---

6. Wiring System

The player must connect the following terminals:

1. Input A → NAND 1 input 1
2. Input B → NAND 1 input 2
3. NAND 1 output → NAND 2 input 1
4. NAND 1 output → NAND 2 input 2
5. NAND 2 output → LED

Wiring interactions

Support two methods.

Click-to-connect

- Click a source terminal.
- Highlight available terminals.
- Click the destination terminal.
- Create a visible wire.

Drag-to-connect

- Press and hold a connection terminal.
- Drag toward another terminal.
- Show a temporary wire following the pointer.

- Release over a valid terminal.
- Complete the connection.

Both methods must work on desktop.

Click-to-connect must also be fully usable on mobile.

Wire appearance

Use SVG paths.

Completed wires should be orange.

Pending connections may use cyan or dashed outlines.

Wires should remain attached to their corresponding terminals when the layout resizes.

Avoid hardcoding wire positions that break at smaller viewport sizes.

Incorrect connections

If the player attempts an invalid connection:

- Do not crash or freeze.
- Provide a short explanation.
- Keep the game state consistent.

The application should distinguish structurally invalid connections from valid connections that produce the wrong circuit.

Allow users to remove or replace incorrect wires.

---

7. Digital Logic Simulation

Implement the NAND operation:

```
function nand(a: boolean, b: boolean): boolean {
  return !(a && b);
}
```

The first NAND gate computes:

```
const x = nand(a, b);
```

The second NAND gate uses X on both inputs:

const output = nand(x, x);

Therefore:

output === (a && b);

Important requirement

This calculation should be driven by the player's wired circuit.

Represent connections using a graph or terminal-to-terminal mapping.

Evaluate the graph from the input sources to the output.

When the circuit is incomplete, show an incomplete or unknown output rather than pretending that a valid circuit exists.

Prevent circular evaluation from causing an infinite loop.

---

8. Input Switches

Provide two interactive inputs:

INPUT A

INPUT B

Each input should toggle between:

- 0 — OFF
- 1 — ON

When the player toggles an input:

- Update the corresponding input value.
- Recalculate the circuit.
- Update connected wire states.
- Update the NAND gate outputs.
- Update the LED when the output is valid.

LED behavior

The LED must reflect the actual output.

When output = 1:

- LED glows orange.
- Status shows `OUTPUT HIGH`.

When output = 0:

- LED remains dark.
- Status shows `OUTPUT LOW`.

When the circuit is incomplete:

- LED remains off.
- Status shows `CIRCUIT INCOMPLETE`.

---

9. Truth Table Testing

Include an interactive truth table.

A	B	Expected	Actual
0	0	0	—
0	1	0	—
1	0	0	—
1	1	1	—

Test behavior

The player should be able to run all four combinations.

For each combination:

1. Update inputs A and B.
2. Evaluate the wired circuit.
3. Compare the actual output with the expected AND output.
4. Mark the row as passed or failed.

Provide a button labeled:

`RUN ALL TESTS`

The application should highlight passing and failing results.

Only award completion when all four combinations pass and the circuit uses the required two-NAND topology.

The test process should not automatically repair incorrect wiring.

---

10. Mission Checklist

Display an educational progress checklist.

Mission: Construct an AND Gate

- Place NAND gate 1.
- Place NAND gate 2.
- Connect input A to NAND gate 1.
- Connect input B to NAND gate 1.
- Connect NAND gate 1 output to both NAND gate 2 inputs.
- Connect NAND gate 2 output to the LED.
- Verify all four truth-table combinations.

Each item should update automatically based on the real application state.

Do not allow players to manually mark tasks complete.

Completion state

When all requirements are satisfied, display:

CIRCUIT COMPLETE

"You successfully constructed an AND gate using only NAND gates!"

Include a subtle completion animation and an option to reset and play again.

---

11. Hint System

Include a button labeled:

`SHOW HINT`

Hints should progressively explain the circuit without immediately revealing every connection.

Suggested sequence:

Hint 1: A NAND gate returns 0 only when both inputs are 1.

Hint 2: The output of the first NAND gate is the opposite of A AND B.

Hint 3: Connecting the same signal to both inputs of a NAND gate turns it into a NOT gate.

Hint 4: Send the first NAND gate's output into both inputs of the second NAND gate.

Use the hints to teach the reasoning behind the construction.

---

12. Game State Management

Define explicit game states:

- `idle`
- `placing`
- `wiring`
- `incomplete`
- `valid`
- `testing`
- `completed`

Track:

- Placed gate components
- Terminal connections
- Selected wiring source
- Current input values
- Gate outputs
- LED output
- Truth-table results
- Hint progression
- Mission completion

Keep the circuit graph separate from presentation state.

Use deterministic evaluation functions so the circuit can be tested independently from the user interface.

---

13. Animation Requirements

Use restrained engineering-style animations.

Recommended effects:

- Components snapping onto the board
- Wires drawing themselves when connected
- Terminals glowing on hover
- Signals traveling along completed connections
- LED illuminating when output is high
- Truth-table rows highlighting when tested
- Completion indicator appearing after success

Animations should communicate circuit state rather than merely decorate the page.

Support `prefers-reduced-motion`.

Avoid excessive flashing, rapid pulsing, or distracting effects.

---

14. Responsive Design

The game must support:

- Desktop: 1440px
- Laptop: 1024px
- Tablet: 768px
- Mobile: 390px
- Narrow mobile: 320px

Desktop

Use a large breadboard workspace with a secondary panel for instructions, circuit status, and truth-table testing.

Mobile

Place the interactive board above the instructions and controls.

Ensure terminals are large enough to tap accurately.

Prioritize click-to-connect and tap-to-connect interaction.

The user must not have to zoom the browser to complete the circuit.

Prevent horizontal overflow and accidental page scrolling during active wire dragging.

---

15. Accessibility

Implement:

- Keyboard-operable input switches
- Keyboard-operable wiring
- Visible focus indicators
- Accessible terminal labels
- Status announcements for circuit changes
- Text alternatives for LED states
- Adequate color contrast
- Reduced-motion support
- Clear error messages

Do not rely on orange and white colors alone to distinguish active and inactive signals.

---

16. Component Architecture

Create a reusable component structure:

```
src/
  components/
    projects/
      nand-lab/
        NandLab.tsx
        Breadboard.tsx
        NandGate.tsx
        CircuitWire.tsx
        InputSwitch.tsx
        OutputLED.tsx
        TruthTable.tsx
```

MissionChecklist.tsx
HintPanel.tsx
nandLogic.ts
nandTypes.ts
nandLab.css

Responsibilities

NandLab.tsx

Main game component and overall state coordination.

Breadboard.tsx

SVG breadboard rendering and component positions.

NandGate.tsx

NAND gate graphics and connection terminals.

CircuitWire.tsx

Wire path rendering and connection interactions.

InputSwitch.tsx

Controls input A and input B.

OutputLED.tsx

Displays evaluated output.

TruthTable.tsx

Displays and tests all four combinations.

MissionChecklist.tsx

Tracks actual progress.

HintPanel.tsx

Provides progressive educational guidance.

nandLogic.ts

Pure functions for NAND evaluation, circuit validation, and truth-table testing.

---

17. Portfolio Integration

The game will be incorporated into my existing blueprint-themed personal portfolio.

Create a reusable exported component:

<NandLab />

The parent portfolio should be able to display it inside an expandable project section.

Collapsed state

Display:

NAND LAB

Digital Logic · Circuit Design · Interactive Learning

Short description:

"An interactive breadboard challenge exploring how universal NAND gates can be combined to construct an AND gate."

Include:

`EXPLORE PROJECT`

Expanded state

When the visitor clicks "EXPLORE PROJECT":

- Expand the project container vertically.
- Push subsequent portfolio content downward.
- Render the interactive NAND Lab experience.
- Preserve the portfolio's existing blueprint grid and styling.
- Allow visitors to collapse the project without navigating away.

The game must not modify the portfolio's global navigation or other project components.

When collapsed, cancel active animations and pointer operations.

---

18. Technical Requirements

- Entirely client-side
- No backend
- No database
- No external AI services
- No authentication
- No real hardware connection
- No external circuit simulation API
- No unnecessary dependencies
- No global CSS pollution
- No broken connections after resizing

Use supplied local assets where appropriate.

Prefer SVG for scalable circuit elements and connections.

Keep the application's logic maintainable and testable.

---

19. Testing Requirements

Implement automated tests for the core simulation.

Logic tests

Verify:

- NAND(0,0) = 1
- NAND(0,1) = 1
- NAND(1,0) = 1
- NAND(1,1) = 0

Verify that the completed two-NAND construction produces:

- AND(0,0) = 0
- AND(0,1) = 0
- AND(1,0) = 0
- AND(1,1) = 1

Interaction tests

Verify:

- Gates can be placed.
- Terminals can be connected.
- Incorrect wiring does not crash the application.
- Wires can be removed.
- Input switches update the circuit.
- LED status matches evaluated output.
- Truth-table tests report accurate results.
- Mission progress corresponds to actual wiring.
- Reset clears circuit state.
- Mobile tap-to-connect works.

---

20. Acceptance Criteria

The implementation is complete when:

1. The breadboard is rendered in the supplied blueprint design style.
2. Players can place two NAND gates.
3. Players can connect terminals with visible wires.
4. Inputs A and B can be independently toggled.
5. Gate outputs are calculated from actual wiring.
6. The LED displays the evaluated circuit output.
7. Incorrect or incomplete circuits are detected.
8. All four truth-table combinations can be tested.
9. The mission checklist updates automatically.
10. The hint system works.
11. Reset restores the initial state.
12. Desktop and mobile interactions are functional.
13. The game operates without a backend.
14. The component can be embedded inside the portfolio without breaking its layout.

---

21. Design and Engineering Quality

This is both an educational project and a software engineering portfolio demonstration.

Prioritize:

Technical correctness: Logic evaluation must reflect the actual circuit graph.

User experience: Visitors should understand how to begin without reading lengthy documentation.

Visual consistency: The application should resemble a technical engineering blueprint.

Interactivity: Visitors must construct and test a circuit themselves.

Maintainability: The simulation should have clearly separated state, rendering, and logic.

Do not deliver a static image, a noninteractive circuit diagram, or a hardcoded animation that always shows success.

The final implementation should provide a complete, playable challenge.

Final Development Instructions

First inspect my existing portfolio codebase and identify its framework, styling system, routing, and project component structure.

Use the existing standalone **NAND Lab HTML game** as the functional reference and the provided NAND gate blueprint image as the visual reference.

Recreate the interaction in React and TypeScript, preserving the original game's mission flow and wiring functionality.

Do not rewrite unrelated parts of my portfolio.

Deliver the complete implementation, including components, circuit evaluation, styling, and tests.

The finished result must work both independently and as an expandable interactive project inside my portfolio.

There are two files this should be an interactive front-end part.